

Semantic Branching Cheat Sheet

Semantic Entropy Clustering for Diverse Agentic Code Generation

Ramtin Sirjani & Paul Moore — Western University — Gen AI 2026

Compute Budget

Both baseline and branching are allowed **250 steps per trajectory**. The baseline runs a single greedy trajectory (250 steps total). Branching spawns up to $B=30$ trajectories, each with the same 250-step cap. Results: baseline 5/10, branching 4/10, **union 7/10** — the methods are complementary.

1 NLI Relevance Scoring (Search Phase)

During the search phase the agent runs bash commands (`grep`, `find`, `cat`) to explore the repo. After *each* step we score how relevant the finding is to the bug.

1.1 Why Not Pure DeBERTa NLI?

DeBERTa NLI is trained on **natural language** sentence pairs, not code-to-bug pairs.

Problem	Result
No shared context between code output and bug description	Entailment ≈ 0 always
Too much shared context (copy-pasted keywords)	Entailment ≈ 1 always

Solution: use the LLM itself to bridge the domain gap via two cheap calls.

1.2 Step 1 — LLM Summarization

Converts raw search output into a natural language sentence.

Summarization Prompt

An AI agent is debugging a software issue. It ran a search command and got this result. In one sentence, what did this search find and why might it be relevant?

Agent's reasoning: {`thought`} $\leftarrow$ 300 chars
Search result (truncated): {`observation`} $\leftarrow$ 500 chars
One-sentence summary of what was found:

temperature=0.0, max_tokens=80

Output example: “Found the Permutation constructor in `permutations.py` which handles cycle notation input.”

1.3 Step 2 — LLM Relevance Rating

Relevance Prompt

Rate how relevant this investigation finding is to the bug below.
Bug: {`problem_statement`} $\leftarrow$ 300 chars
Finding: {`finding_summary`} $\leftarrow$ from Step 1
Rate relevance from 0 (completely irrelevant) to 10 (directly identifies the bug).
Respond with **ONLY** a single number 0–10.

temperature=0.0, max_tokens=5

Post-processing: regex extracts first integer $\rightarrow$ clamp to $[0, 10] \rightarrow$ divide by 10 $\rightarrow$ score $\in [0, 1]$.

1.4 Why Not DeBERTa NLI for Relevance?

We tested using DeBERTa entailment instead of LLM-as-judge (even with the LLM summary bridge). Results on `sympy-16766`:

Summary	NLI	LLM
“Found code generation utilities”	0.003	0.7
“Found printing functionality files”	0.038	0.7
“Found Python code printers module”	0.618	0.7
“Found Indexed class docs”	0.005	0.7
“mpmath missing, import failed”	0.017	0.3

Why? NLI measures *logical entailment* (“does A imply B?”), not *topical relevance*. “Found Indexed class” doesn’t logically imply “Indexed printing is broken” — but it’s clearly relevant. NLI works for **clustering** (same-structure intent comparisons) but not for relevance scoring.

1.5 Saturation Detection

Saturation Rule

$$\text{consec_low} \geq N_{\text{cap}} \implies \text{Strategy Phase}$$

A step is “low relevance” if score $\leq \theta$ (default $\theta = 0.5$).

2 Generating 5 Diverse Strategies

After search saturation: (1) build a **search report**, then (2) prompt for strategies.

2.1 Search Report Prompt

Search Report Prompt

An AI coding agent investigated a software bug. Below is its exploration history with the actual code it found.
Write a structured report with these sections:
1. ROOT CAUSE: What is the bug and why?
2. RELEVANT CODE: Verbatim snippets with file/function/line.
3. FIX POINTS: Different places a fix could be applied.
IMPORTANT: Include actual code from observations. Do NOT paraphrase.
Exploration history: {`history`} $\leftarrow$ last 15 relevant steps, 8k chars

temperature=0.0, max_tokens=1500

2.2 Strategy Proposal Prompt (Verbatim)

Strategy Proposal Prompt

A software bug needs to be fixed. Here is the problem description and investigation findings.
Problem
{`problem_statement`} $\leftarrow$ 2000 chars
Investigation Findings
{`search_report`} $\leftarrow$ 4000 chars
Task
Propose exactly {`n`} **FUNDAMENTALLY DIFFERENT** code-level strategies to fix this bug.
CRITICAL RULES:

- Each strategy **MUST** take a **COMPLETELY DIFFERENT APPROACH**
- Each strategy **MUST** modify **DIFFERENT** lines, functions, or files
- Each strategy **MUST** produce a structurally different patch
- Do NOT propose strategies that all change the same line
- Be **SPECIFIC**: name exact function, file, and line range
- Describe the actual code transformation

Think about fundamentally different approaches:

- Fix at different levels of the call stack
- Fix in different files or modules
- Use different mechanisms (validation vs normalization vs delegation)
- Address different root causes

IMPORTANT: Before writing each strategy, explicitly state what makes it **FUNDAMENTALLY DIFFERENT** from all previous strategies.
Format: STRATEGY 1: [`file/function`] -- [`specific code change`]

3.5 Semantic Entropy

Semantic Entropy (Discrete Variant)

$$H = - \sum_{k=1}^K p(c_k) \cdot \ln(p(c_k))$$

where $p(c_k) = \frac{|c_k|}{N}$

Every variable:

Sym.	Meaning
H	Semantic entropy — diversity in nats
K	Number of semantic clusters
k	Cluster index, 1 to K
c_k	Cluster k (set of candidate indices)
$ c_k $	Size of cluster k
N	Total candidates ($\sum_k c_k $)
$p(c_k)$	Cluster probability = $ c_k /N$
$\ln$	Natural log (base- e)

Bounds:

Scenario	Sizes	H
All in one cluster	[5]	0
Two equal groups	[3, 2]	0.673
All singletons	[1, 1, 1, 1, 1]	$\ln 5 \approx 1.609$

Min: $K=1 \Rightarrow p=1 \Rightarrow -1 \cdot \ln 1 = 0$

Max: K equal clusters $\Rightarrow H = \ln K$

3.6 Branching Decision

Branch Criterion

$$\text{branch} = [H > \tau] \quad \text{default } \tau = 0.5$$

- $H > \tau$: diverse $\rightarrow$ **branch** — each cluster spawns a trajectory
- $H \leq \tau$: converged $\rightarrow$ **no branch** — take greedy action

3.7 Worked Example

5 strategies, clustered with $\theta = 0.5$:

	Strategy
s_0	Normalize overlapping cycles in constructor
s_1	Raise ValueError when cycles share elements
s_2	Canonicalize cycle input in from_sequence
s_3	Add validation in Permutation constructor
s_4	Fix _af_new helper for overlapping cycles

Clustering trace:

i	fwd	bwd	Result
0	—	—	New cluster $A=\{0\}$
1	.12	—	$< \theta \rightarrow$ new $B=\{1\}$
2	.78	.65	Both $> \theta \rightarrow A=\{0, 2\}$
3	.72	.58	Both $> \theta \rightarrow A=\{0, 2, 3\}$
4	.21	—	$< \theta$ for s_0 , .15 for $s_1 \rightarrow$ new $C=\{4\}$

Result: 3 clusters, sizes [3, 1, 1]

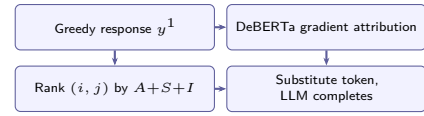
Entropy:

$$\begin{aligned}
 p(A) &= \frac{3}{5} = 0.6, \quad p(B) = p(C) = \frac{1}{5} = 0.2 \\
 H &= -[0.6 \ln(0.6) + 0.2 \ln(0.2) + 0.2 \ln(0.2)] \\
 &= -[0.6(-0.511) + 0.2(-1.609) + 0.2(-1.609)] \\
 &= -(-0.306 - 0.322 - 0.322) \\
 &= \mathbf{0.950}
 \end{aligned}$$

Decision: $0.950 > 0.5 = \tau \Rightarrow$ **BRANCH** into 3 trajectories.

4 SDLG — Semantically Diverse Language Generation

Aichberger et al. (2025). Generates diverse alternatives by gradient-based token attribution through DeBERTa, then substitution + LLM re-completion.



4.1 Step 0: Greedy Response

Generate baseline at temp = 0. Split into:

Part	Contents
THOUGHT	Natural language reasoning
CODE	Bash command in "mswea_bash_command"

4.2 Step 1: Attribution Score A_i

Goal: Which tokens carry the most semantic weight?

4.2.1 Setup — Self-Entailment

Feed text as **both** premise and hypothesis:

$$[\text{CLS}] \underbrace{\text{text}}_{\text{prem.}} [\text{SEP}]^2 [\text{SEP}]^2 [\text{SEP}]^2 [\text{SEP}]^2 \underbrace{\text{text}}_{\text{hyp.}} [\text{SEP}]$$

4.2.2 Loss — Target Contradiction

Contradiction Loss

$$\mathcal{L} = \text{CE}(l, \text{target}=0) = -\log \frac{\exp(l_0)}{\sum_k \exp(l_k)}$$

Why contradiction? The text trivially entails itself. The gradient of the contradiction loss reveals which tokens, if changed, would flip self-entailment to self-contradiction — i.e., the tokens that carry the meaning.

4.2.3 Backprop & Attribution

Backpropagate $\mathcal{L}$ through the embedding layer to get $\nabla_{z_i} \mathcal{L}$ for each token i . Extract **hypothesis portion only**.

Attribution Score

$$A_i = \|z_i \odot \nabla_{z_i} \mathcal{L}\|_2$$

Every symbol:

Sym.	Meaning	Shape
i	Token position in hypothesis	int
z_i	Embedding vector of token i	$[D]$
D	Embedding dim (1024 for DeBERTa-large)	int
$\mathcal{L}$	Contradiction loss	scalar
$\nabla_{z_i} \mathcal{L}$	Gradient of $\mathcal{L}$ w.r.t. z_i	$[D]$
$\odot$	Hadamard (element-wise) product	$[D]$
$\ \cdot\ _2$	L2 norm: $\sqrt{\sum_d (\cdot)^2}$	scalar
A_i	Attribution score for token i	≥ 0

Intuition for each piece:

Component	What it captures
z_i	Large embedding = strong signal
$\nabla_{z_i} \mathcal{L}$	Large gradient = loss sensitive to this token
$z_i \odot \nabla$	Both conditions simultaneously
$\ \cdot\ _2$	Collapses D dims to one scalar

Normalization: $A_i \leftarrow A_i / \max_i(A_i)$ so $A_i \in [0, 1]$.

Word boundary filter: Only word-initial BPE tokens (prefix Ġ or _) are candidates. Avoids sub-word fragments.

4.3 Step 2: Substitution Score S_{ij}

Goal: For position i , which replacement j from the vocabulary shifts meaning most along the gradient?

Substitution Score
$S_{ij} = \frac{(z_i - z_j) \cdot \nabla_{z_i} \mathcal{L}}{\ z_i - z_j\ _2 \cdot \ \nabla_{z_i} \mathcal{L}\ _2}$

Every symbol:

Sym.	Meaning
z_i	Embedding of original token at pos i
z_j	Embedding of candidate replacement j
$z_i - z_j$	How much the embedding changes
$\nabla \mathcal{L}$	Direction increasing contradiction
$\cdot$	Dot product
S_{ij}	Cosine sim. of change vs gradient

Interpretation:

S_{ij}	Meaning
$\approx +1$	Moves along gradient $\rightarrow$ max meaning change
≈ 0	Orthogonal $\rightarrow$ no semantic shift
≈ -1	Against gradient $\rightarrow$ reinforces original

Computed **server-side** over the full vocabulary ($V \approx 128k$):

```
diff = z_i.unsqueeze(0) - emb_matrix # [V, D]
diff_norms = diff.norm(dim=-1) # [V]
s_ij = (diff @ grad_i) / (diff_norms * grad_norm)
```

Top- K ($K=20$) replacements selected. Self-substitution excluded.

4.4 Step 3: Importance Score I_{ij}

Goal: Is the substitute plausible under the LLM?

Importance Score
$I_{ij} = p(v_j \mid y_{<i}, x, w)$

Symbol	Meaning
v_j	Candidate replacement token
$y_{<i}$	All text before position i
x	Full conversation context (history)
w	LLM weights (Qwen3-Coder-30B)
$p(\cdot)$	LLM's next-token distribution

Computed via **vLLM**: send prefix to `/v1/completions` with `logprobs=20`, convert: $I_{ij} = \exp(\text{logprob}_j)$.

Why I_{ij} matters

Without I_{ij} , SDLG might substitute “remove” $\rightarrow$ “xylophone” (high S_{ij} because embeddings are far apart, but nonsensical). I_{ij} filters for tokens the LLM would actually generate.

4.5 Step 4: Combined Ranking

Combined Score
$\text{Combined}_{ij} = \frac{A_i + S_{ij} + I_{ij}}{3}$

	Range	Measures	Source
A_i	$[0, 1]$	Position importance	DeBERTa ∇
S_{ij}	$[-1, 1]$	Meaning shift	emb + ∇
I_{ij}	$[0, 1]$	LLM plausibility	vLLM

All (i, j) pairs sorted by Combined_{ij} descending.

4.6 Step 5: Prefix Truncation & Completion

4.6.1 THOUGHT-Level (Strategic Diversity)

Changes the *reasoning*, causing a different fix approach:

Thought Substitution

1. Find original token in thought text
2. Replace with substitute
3. **Truncate everything after** substitution point
4. Send truncated thought as **assistant** message pre-fix
5. LLM regenerates rest of reasoning **AND** code

Example:

- Greedy: “I should **remove** the duplicate check...”
- Substitute: “remove” $\rightarrow$ “compose”
- Prefix: “I should compose”
- LLM completes: “... the cycles before validation...” + *new code*

4.6.2 CODE-Level (Tactical Diversity)

Keeps reasoning fixed, diversifies the implementation:

Code Substitution

1. Keep thought text **unchanged**
2. Find original token in code block
3. Replace with substitute, truncate
4. LLM completes the code from there
5. Result: same reasoning, different patch

4.6.3 Budget Split

For N total candidates (default $N=5$):

Source	Count
Greedy response (always kept)	1
THOUGHT-level SDLG	$\lceil (N-1)/2 \rceil = 2$
CODE-level SDLG	$\lfloor (N-1)/2 \rfloor = 2$

4.7 Fallback

If SDLG produces zero alternatives $\rightarrow$ **temperature sampling** at $T=0.7$.